

一、项目概述

本次开发项目为基于 ****Vue3 + Express + PostgreSQL**** 技术栈的校园社区平台，核心目标是实现校内用户的发帖互动、附件共享、分类交流等功能。我主要负责帖子核心业务链路的全栈开发，覆盖发帖功能、帖子详情页、附件上传体系、权限控制、交互体验优化以及本地局域网部署适配等工作。截至目前，已完成三个阶段的功能迭代，打通了从内容发布、展示、互动到本地部署的完整链路，功能均达到可用标准。

二、已完成工作与个人贡献

（一）发帖功能模块落地

作为帖子业务的起点，完成了发帖功能的前端开发、接口联调与版本管理：

1. ****前端页面开发****：基于 **Vue3 + Element Plus** 开发 ``WritePostView.vue`` 发帖页面，严格遵循 **Apifox** 接口契约设计表单，仅保留 ``title``、``content``、``categoryId``、``tags`` 四个规范字段；实现必填项校验、标签格式转换逻辑，将用户输入的逗号分隔字符串转换为后端约定的数组格式。
2. ****接口与路由封装****：独立封装发帖接口请求模块，统一管理请求路径与参数格式；完成页面路由注册与全局导航入口适配，完善项目跳转链路。
3. ****前后端联调****：梳理跨域、代理、请求格式等配置，打通前端到 **PostgreSQL** 数据库的完整数据链路，实现发帖数据持久化。
4. ****版本管理****：按照团队规范基于主分支创建 ``feature.post`` 功能分支，筛选有效业务代码、过滤冗余文件，完成提交并推送至远程仓库，保障协作规范。

（二）帖子详情页与附件体系搭建

围绕帖子详情场景，完成内容渲染、附件上传、权限控制等核心能力：

1. ****详情页核心渲染与认证联动****

严格对照接口协议实现帖子全字段渲染，覆盖标题、正文、作者信息、发布时间、标签、互动统计等核心元素；对接项目已有 **auth** 认证模块，通过 ``getMe`` 接口拉取当前登录用户身份，建立页面级权限上下文，实现登录态与功能入口的联动，未登录状态自动隐藏操作按钮。

2. ****多类型附件上传体系****

- 前端：封装统一的上传交互组件，支持图片、文档、视频三类主流文件格式；实现 **100MB** 文件大小前置校验与格式拦截，完善文件选取、上传状态反馈的完整交互流程。
- 后端：适配多类型文件接收逻辑，配置静态资源托管目录，完成文件存储、访问路径返回与下载链路打通；补充服务端文件类型与体积二次校验，保障上传安全。
- 数据层：设计独立的附件信息数据表，存储文件名称、存储路径、文件类型、大小、关联帖子 ID 等元数据，替代帖子表直接嵌套的方案，降低数据冗余；同步更新数据库设计文档

并对齐团队数据规范。

3. ****双重权限控制机制****

实现帖子删除的前后端双重校验：前端基于当前登录用户 ID 与帖子作者 ID 做匹配，结合用户角色字段判断管理员身份，仅本人与管理员可见删除按钮；后端接口同步新增权限校验层，二次拦截越权请求，保障操作安全合规。

4. ****分类标签系统优化****

梳理分类筛选的参数传递逻辑，统一标签在列表页、详情页等多场景下的渲染与解析规则，修复字段格式不一致导致的筛选异常、展示错乱问题，提升前端交互准确性。

（三）功能迭代与部署能力拓展

在基础功能之上，完成体验优化、机制升级与本地部署落地：

1. ****多格式附件在线预览****

实现图片、PDF、视频三类格式的原生内嵌预览：图片支持大图弹窗查看，PDF 采用 iframe 内嵌渲染，视频支持原生控件播放；针对 docx 等 Office 格式设计双方案适配，当前版本采用降级策略，识别到不支持预览的格式自动触发下载，兼顾格式覆盖度与功能可用性。

2. ****帖子软删除机制落地****

将原物理删除逻辑改造为时间戳标记删除模式，在数据库新增删除标识字段；同步改造所有帖子查询接口，新增过滤条件默认排除已删除数据，在前端用户无感知的前提下，保留了数据可恢复性与操作审计能力。

3. ****交互体验优化****

实现帖子列表页到详情页的分页记忆跳转，通过 URL query 参数传递分页状态，详情页返回时携带原页码参数，列表页自动定位到对应分页，避免用户跳转后重复翻页；修复附件路径解析、格式识别等边界场景 BUG，补充异常场景的友好提示，提升页面容错性。

4. ****局域网部署与多设备互通****

完成全流程本地部署适配：后端监听地址改为 `0.0.0.0` 放开局域网访问权限；通过 `BASE\_URL` 环境变量统一生成附件访问地址；实现前端打包后由后端单端口托管，配套 history 路由兜底逻辑；配合 Windows 防火墙入站规则配置，最终实现同校园 WiFi 环境下多设备全功能互通，验证了账号登录、发帖评论、附件上传下载的完整链路。

三、问题解决与 AI 辅助开发实践

开发过程中横跨发帖功能、详情页体系、附件上传、权限控制、部署落地多个阶段，遇到了大量技术与工程问题。我始终坚持****「自主判断为核心，AI 为效率工具」****的协作原则：所有问题均先基于自身技术积累独立分析、定位方向、尝试解决，遇到思路瓶颈或效率瓶颈时，再将 AI 作为技术参谋，用于补充知识盲区、提供系统化排查框架、给出通用备选方案。最终的方案设计、代码实现、业务适配均由自己主导完成，绝非直接照搬 AI 输出。这种模式既避免了无差别滥用 AI 导致的代码与项目脱节问题，又大幅提升了疑难问题的解决效率，同时也在过程中验证并强化了自身的技术能力。

1. 受保护业务接口本地调试问题

****自主分析与初步尝试**:**

开发发帖功能初期，系统业务接口已接入 JWT 身份鉴权，但完整的用户注册登录体系尚未落地，导致发帖等受限接口无法正常调试。我首先尝试了直接在请求头硬编码 Token 的方案，但 Token 有效期短且构造复杂，调试效率很低。我已经意识到需要构造一个稳定的模拟身份上下文，但如何在不破坏正式鉴权逻辑的前提下实现本地开发豁免，还没有最优方案。

****AI 辅助与方案优化**:**

咨询 AI 后，它给出了「开发环境临时身份模拟」的思路，提供了中间件豁免、模拟用户注入两种备选方案。我评估后认为中间件豁免会破坏鉴权体系的完整性，于是选择了模拟管理员用户的方案：基于项目已有的 JWT 规范，自行构造合法的登录凭证与用户身份上下文，在本地开发环境绕过完整登录流程。

我没有直接使用 AI 生成的模拟代码，而是结合项目已有的 `auth.js` 模块结构，将模拟身份逻辑做成了可配置的开发工具函数，通过环境变量控制开关，生产环境自动失效，既保障了调试效率，又不影响正式鉴权逻辑的完整性。

****最终效果**:** 所有受保护接口的本地开发与调试顺利推进，无需等待完整认证体系落地，大幅加快了发帖功能的开发进度。

2. 前后端通信链路连通性问题

****自主分析与初步尝试**:**

前后端联调初期，服务端接口频繁访问失败，浏览器控制台报跨域错误。我首先自主排查了 `cors` 中间件配置、请求地址端口、后端服务启动状态三个常见问题，确认基础配置无误，但问题依然存在。初步判断问题出在 Vite 开发服务器的代理转发环节，但由于对代理的路径匹配规则不熟悉，逐个试错效率较低。

****AI 辅助与方案优化**:**

我将报错现象与已排查结论同步给 AI，它输出了一套「**跨域配置 - 代理规则 - 路由注册**」三层排查框架，帮我把零散的排查点梳理成了系统化的链路。我按照这个框架逐层验证，很快定位到根因：Vite 代理的前缀匹配规则配置偏差，导致长路径请求未正确转发到后端，而是被前端路由拦截了。

AI 给出了通用的代理配置示例，我没有直接复制，而是结合项目已有的代理配置结构，调整了路径重写规则与匹配优先级，同时补充了代理超时配置，最终的方案更贴合项目现有架构，也更具可维护性。

****最终效果**:** 前后端通信链路稳定打通，所有接口请求正常响应，这套排查思路也被沿用后续所有功能的联调中。

3. 接口字段定义与契约不一致问题

****自主分析与初步尝试**:**

发帖功能联调时，后端持续返回参数校验失败。我对比了 Apifox 接口文档，发现前端请求体多出了几个自行扩展的字段，同时部分字段命名与约定格式存在偏差。我已经意识到需要以接口契约为唯一标准裁剪请求体，但如果每个接口都单独处理字段，代码冗余度高且后续

维护困难。

****AI 辅助与方案优化**:**

AI 强调了「接口契约唯一标准」原则，给出了在每个接口单独裁剪字段的通用方案。我认为这种方式复用性差，不符合项目的工程化要求，于是结合项目已有的 `axios` 请求封装，自行设计了统一的请求参数拦截器：在全局请求层面自动过滤非契约字段，同时按约定规则统一转换字段格式。

这个方案比 AI 给出的逐接口处理方案更具通用性，后续所有新增接口都自动遵循契约规范，从机制上避免了字段偏差问题，而不是靠人工逐个校验。

****最终效果**:** 所有业务接口参数完全对齐团队规范，请求体格式统一，从根源上减少了接口联调的字段适配问题。

4. 帖子详情页多接口耦合与认证联动问题

****自主分析与初步尝试**:**

开发帖子详情页时，需要同时对帖子详情、用户信息、互动状态、评论列表多个业务接口，还要接入登录认证做权限判断，多环节耦合，调试链路很长。我采用了分步联调的思路，先打通基础详情接口，再逐步叠加认证与权限逻辑，但在身份信息与页面功能的联动上，出现了登录态获取延迟导致的按钮显隐错乱问题。

****AI 辅助与方案优化**:**

我将多接口耦合的调试痛点告知 AI，它给出了「接口分层加载 + 权限上下文统一管理」的优化思路，建议将用户身份信息抽成全局状态，避免每个页面重复拉取。我参考这个思路，结合项目已有的 `auth` 认证模块，建立了页面级的权限上下文：通过 `getMe` 接口统一拉取当前登录用户身份，在组件外层做权限计算，内层业务组件直接消费权限结果。

同时，我自行补充了加载态处理与边界场景判断：未登录状态自动隐藏所有操作入口，身份加载过程中按钮置灰禁用，避免了权限判断闪烁与越权操作的可能。这些细节处理都是基于自身的理解补充的，AI 并未覆盖到。

****最终效果**:** 详情页与登录系统完整联动，权限判断准确，交互状态流畅，未再出现身份与功能不同步的问题。

5. 多类型文件上传全链路稳定性问题

****自主分析与初步尝试**:**

从单图片上传拓展到图片、文档、视频三类附件时，集中爆发了大量问题：100MB 大文件传输超时、部分文件类型识别错误、存储路径混乱。我最初针对每个单点问题逐个修复：调整了 `multer` 的大小限制、补充了前端类型判断，但很快发现治标不治本——附件路径直接存储在帖子表中，数据冗余高，后续扩展预览、权限控制都会非常困难。我已经意识到需要拆分数据表重构附件管理，但完整的表结构设计与接口改造方案还需要系统参考。

****AI 辅助与方案优化**:**

我向 AI 描述了全链路故障与我的数据层重构思路后，它首先肯定了拆分表的方向，然后给

出了「沿上传全链路五节点排查」的方法，帮我系统性梳理了前端校验、网络传输、后端解析、磁盘存储、路径返回每个环节的常见故障点。我对照这个清单，补全了前端 100MB 大小前置拦截、后端文件类型二次校验等遗漏的校验节点。

在数据表设计上，AI 给出了通用的附件表字段建议，我没有直接套用，而是结合项目已有的 PostgreSQL 设计规范、帖子表的主键规则，自行设计了 `attachments` 表结构，定义了与帖子表的外键关联与索引规则，同时同步更新了数据库设计文档。整个改造的业务逻辑、SQL 实现、接口适配均由自己独立完成。

****最终效果****：三类附件上传成功率稳定达标，文件元数据管理规范，数据层结构清晰，为后续附件预览、删除权限等功能打下了良好的数据基础。

6. 帖子删除双重权限控制实现问题

****自主分析与初步尝试****：

实现帖子删除功能时，要求仅帖子本人与管理员可执行操作。我首先明确了前后端双重校验的设计原则——前端做按钮显隐控制体验，后端做权限拦截保安全。前端的作者 ID 匹配逻辑比较容易实现，但如何可靠获取当前用户角色、如何在后端统一做权限校验，还需要结合现有认证体系设计方案。

****AI 辅助与方案优化****：

咨询 AI 后，它给出了「前端角色判断 + 后端中间件校验」的通用实现方案。我参考这个框架，复用了项目已有的 `auth.js` 认证模块与 `getMe` 接口，提取用户 ID 与角色字段，在前端实现了「作者 ID 匹配 + 管理员角色判断」双逻辑的按钮显隐控制。

后端部分，我没有直接使用 AI 生成的权限中间件代码，而是结合项目现有的 JWT 鉴权中间件，新增了权限校验逻辑：先从 Token 中解析用户身份，再查询帖子作者信息做匹配，同时校验管理员角色，双重校验通过才允许执行删除操作。整个实现完全贴合项目现有鉴权体系，没有引入额外的依赖与冗余代码。

****最终效果****：删除权限控制准确可靠，越权操作被有效拦截，符合平台的安全设计要求。

7. Vue History 模式静态托管页面 404 问题

****自主分析与初步尝试****：

前端打包后由后端托管时，根路径与子页面刷新均返回 `Cannot GET /` 报错。我首先排除了最常见的路径配置错误：通过打印绝对路径确认了 `dist` 目录定位准确，手动访问 `index.html` 文件也能正常读取，由此判断问题本质是 Vue Router 的 history 模式需要后端支持 URL 重写，而纯静态托管做不到。

我最初尝试用 `app.get('\*')` 通配符路由做兜底，这是行业常规写法，但运行后出现了 `path-to-regexp` 库的版本兼容报错。我通过分析报错栈，自行定位到是当前 Express 依赖版本下，通配符 `\*` 的解析规则与预期不一致，需要换一种实现方式。

****AI 辅助与方案优化****：

带着「通配符路由版本不兼容」这个已定位的结论，我向 AI 咨询替代方案，它给出了中间件兜底的实现思路。我在这个思路的基础上做了两处关键的业务化改进：第一，新增了 `/api`

接口路径与 `/static` 静态资源路径的排除逻辑，确保兜底只作用于前端路由，不会拦截业务接口；第二，增加了请求方法判断，只对 GET 请求做页面兜底，避免影响 POST、PUT 等业务请求。

最终落地的方案，比 AI 给出的通用版本安全性更高、适配性更强，完全贴合项目的路由结构与接口规范。

****最终效果****：彻底解决了 history 模式下的页面 404 问题，子页面刷新、直接访问均正常，同时完全不影响接口与静态资源的响应。

8. 局域网多设备部署全链路连通性问题

****自主分析与初步尝试****：

要实现同校园 WiFi 下多设备访问互动，我首先知道核心是修改后端监听地址，把 `127.0.0.1` 改成 `0.0.0.0`，但修改后外部设备依然无法访问。我初步判断是 Windows 防火墙的问题，但手动配置规则后依然不通，同时还出现了附件下载地址指向本地、页面资源路径错误等连锁问题。我缺乏系统的局域网部署经验，零散排查效率很低。

****AI 辅助与方案优化****：

AI 先帮我梳理了局域网访问的完整技术链路，纠正了我对「监听地址」与「访问地址」的认知偏差，然后给出了四步部署框架：放开监听 → 统一资源地址 → 前端打包托管 → 防火墙配置。更有价值的是，它提供了「**网络层 - 端口层 - 应用层 - 资源层**」四层排查方法论，让我从零散试错变成了系统化定位。

我按照这个体系逐层验证：先用 `ping` 确认网络连通，再用 `telnet` 验证端口状态，很快定位到是防火墙规则没有生效，重新配置入站规则后端口成功打通。针对附件地址错误的问题，我提出了用 `BASE\_URL` 环境变量统一管理的工程化方案，AI 给予了肯定并补充了调用规范，我据此改造了所有接口的资源路径返回逻辑。

****最终效果****：同校园 WiFi 下所有设备均可通过局域网 IP 正常访问平台，账号、帖子、评论、附件全链路数据同步互通，达到了本地部署的可用标准。

实践总结

回顾整个开发过程，AI 对我而言始终是****效率工具与技术参谋****，而非替代开发者的角色。它的核心价值体现在三个方面：一是提供系统化的问题排查框架，帮我把零散的经验梳理成体系，减少单点试错的时间；二是补充知识盲区，在我不熟悉的领域给出通用方案与方向参考，避免从零开始摸索；三是减少重复劳动，常规语法、通用写法可以快速生成参考，把精力放在业务逻辑与架构设计上。

而所有方案的最终决策权、代码实现的主导权、业务适配的判断权，始终掌握在自己手中。我始终认为，用好 AI 的前提是自身具备足够的技术判断力——能判断 AI 方案的优劣、能识别通用方案的局限性、能结合项目实际做适配改造。未来我也会继续保持这种「自主主导、AI 赋能」的模式，在提升开发效率的同时，持续强化自身的技术能力与工程思维。

四、未来工作规划与团队贡献

1. 核心功能迭代深化

- 完善附件预览能力：落地 Office 文档在线预览方案，扩展支持的文件格式范围，进一步提升附件查看体验；
- 完善软删除管理能力：开发管理员查看、恢复已删除帖子的功能入口，强化平台内容管理能力；
- 深化评论互动体系：优化评论层级展示、回复提醒等功能，补全社区互动链路。

2. 体验与性能优化

- 页面性能优化：探索静态资源缓存、图片懒加载、接口请求优化等方案，提升页面加载与访问速度；
- 边界场景完善：补充更多异常状态的操作反馈与引导提示，提升页面容错性与用户体验。

3. 团队协作与规范建设

- 同步功能变更：与组内成员对齐附件表结构、软删除字段规范、局域网部署方案等变更点，完成跨分支代码兼容性排查，保障项目整体一致性；
- 技术沉淀分享：整理开发中遇到的典型问题与解决方案，形成开发手册与问题排查文档，分享给团队成员，提升整体开发效率；
- 代码质量把控：参与团队代码评审，校验接口字段一致性、权限逻辑完整性与边界场景处理，共同保障项目代码质量。

4. 部署能力拓展

- 优化局域网部署体验：探索 IP 动态适配、一键启动脚本等方案，降低本地部署门槛，方便团队成员快速调试；
- 预研公网部署方案：为后续项目上线提前调研服务器部署、域名配置、生产环境运维等相关方案，做好技术储备。